



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

FRANCISCO WERLEY DA SILVA

**APLICAÇÃO DE MÉTRICAS ESTRUTURAIS PARA AVALIAÇÃO DE
MODULARIDADE EM SISTEMAS ORIENTADOS A OBJETOS: UM ESTUDO
COMPARATIVO ENTRE ARQUITETURA EM CAMADAS E ARQUITETURA
HEXAGONAL**

QUIXADÁ
2026

FRANCISCO WERLEY DA SILVA

APLICAÇÃO DE MÉTRICAS ESTRUTURAIS PARA AVALIAÇÃO DE MODULARIDADE
EM SISTEMAS ORIENTADOS A OBJETOS: UM ESTUDO COMPARATIVO ENTRE
ARQUITETURA EM CAMADAS E ARQUITETURA HEXAGONAL

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Engenharia de Software
do Campus Quixadá da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Engenharia de Software.

Orientador: Prof. Dr. Emanuel Coutinho.

QUIXADÁ

2026

RESUMO

A modularidade é um dos principais atributos de qualidade em sistemas de software, estando diretamente relacionada à manutenibilidade, compreensão e evolução das aplicações. Nesse contexto, as decisões arquiteturais exercem papel fundamental na organização estrutural dos sistemas, influenciando aspectos como acoplamento, coesão e dependência entre componentes. No entanto, a escolha de estilos arquiteturais, como arquitetura em camadas e arquitetura hexagonal, frequentemente é baseada em boas práticas e experiência, sem uma análise quantitativa consistente de seus impactos na qualidade estrutural do software.

Diante disso, este trabalho tem como objetivo avaliar a modularidade de sistemas orientados a objetos por meio da aplicação de métricas estruturais em diferentes estilos arquiteturais. Para isso, é realizado um estudo experimental comparativo utilizando dois sistemas previamente desenvolvidos, cada um representando um estilo arquitetural distinto. A análise é conduzida com base em métricas estruturais clássicas, relacionadas a acoplamento, coesão e complexidade, permitindo uma avaliação quantitativa da qualidade estrutural dos sistemas.

Espera-se, com este trabalho, contribuir para uma melhor compreensão do impacto das decisões arquiteturais na modularidade do software, fornecendo subsídios para a adoção de práticas mais fundamentadas na Engenharia de Software.

Palavras-chave: Modularidade; Arquitetura de Software; Métricas de Software; Sistemas Orientados a Objetos.

ABSTRACT

Modularity is a key quality attribute in software systems, being directly related to maintainability, understandability, and evolution. In this context, architectural decisions play a fundamental role in the structural organization of systems, influencing aspects such as coupling, cohesion, and dependencies among components. However, the selection of architectural styles, such as layered architecture and hexagonal architecture, is often based on best practices and prior experience, without a consistent quantitative evaluation of their impact on software structural quality.

Therefore, this work aims to evaluate the modularity of object-oriented systems through the application of structural metrics across different architectural styles. To achieve this, a comparative experimental study is conducted using two previously developed systems, each representing a distinct architectural approach. The analysis is based on classical structural metrics related to coupling, cohesion, and complexity, enabling a quantitative assessment of the systems' structural quality.

This work is expected to contribute to a better understanding of how architectural decisions affect software modularity, providing support for more evidence-based practices in Software Engineering.

Keywords: Modularity; Software Architecture; Software Metrics; Object-Oriented Systems.

LISTA DE FIGURAS

Figura 1 – Representação do padrão arquitetural em camadas	11
Figura 2 – Representação do padrão arquitetural hexagonal	13
Figura 3 – Fluxo das etapas do procedimento experimental	31

LISTA DE TABELAS

Tabela 1 – Comparativo de abordagens dos trabalhos relacionados	24
Tabela 2 – Caracterização dos sistemas analisados	28
Tabela 3 – Métricas utilizadas na avaliação da modularidade	30
Tabela 4 – Cronograma de execução do TCC II	32

SUMÁRIO

1	INTRODUÇÃO	8
1.1	Objetivos	9
1.1.1	<i>Objetivo Geral</i>	9
1.1.2	<i>Objetivos Específicos</i>	9
2	FUNDAMENTAÇÃO TEÓRICA	10
2.1	Arquitetura de Software	10
2.1.1	<i>Arquitetura em Camadas</i>	11
2.1.2	<i>Arquitetura Hexagonal</i>	12
2.2	Modularidade em Software	14
2.2.1	<i>Acoplamento</i>	15
2.2.2	<i>Coesão</i>	16
2.2.3	<i>Complexidade</i>	17
3	TRABALHOS RELACIONADOS	18
3.1	Avaliação de Conectividade em Sistemas Orientados por Objetos	18
3.2	Os efeitos da arquitetura dirigida a eventos na modularidade de software	19
3.3	Comparação entre as arquiteturas de software monolítica e de microser- viços	20
3.4	Análise comparativa entre arquiteturas de software: monolítica, modular e microsserviços	21
3.5	Um estudo sobre a relação entre qualidade e arquitetura de software . .	22
3.6	Síntese e lacunas	23
4	METODOLOGIA	26
4.1	Classificação da Pesquisa	26
4.2	Estratégia de Pesquisa	26
4.3	Objetos de Estudo	27
4.4	Coleta dos Dados	28
4.5	Métricas de Avaliação	29
4.6	Procedimento Experimental	30
4.7	Cronograma de Execução	31
5	RESULTADOS	33
6	CONCLUSÕES	34

	REFERÊNCIAS BIBLIOGRÁFICAS	35
	APÊNDICE A – Ferramentas e Repositórios Utilizados na Pesquisa . .	36
A.1	Ferramenta de Extração de Métricas	36
A.2	Repositórios dos Sistemas Analisados	36

1 INTRODUÇÃO

A Engenharia de Software tem como objetivo o desenvolvimento de sistemas que atendam não apenas aos requisitos funcionais, mas também a atributos de qualidade que garantam sua sustentabilidade ao longo do tempo. Entre esses atributos, a manutenibilidade destaca-se como um dos mais relevantes, uma vez que grande parte dos custos de um sistema está associada à sua evolução e manutenção ao longo do ciclo de vida (Pressman, 2010). Nesse contexto, a modularidade é amplamente reconhecida como um fator fundamental para a melhoria da qualidade estrutural de sistemas de software, pois permite a decomposição do sistema em módulos menores e independentes, possibilitando que eles sejam modificados de forma isolada e sem que seja preciso conhecer profundamente a implementação de outros módulos (Parnas, 1972).

A modularidade está diretamente relacionada a conceitos como coesão e acoplamento. Sistemas com alta coesão e baixo acoplamento tendem a apresentar melhor organização estrutural, facilitando sua compreensão, teste e modificação (Yourdon; Constantine, 1978). Em sistemas orientados a objetos, essas características são fortemente influenciadas pelas decisões arquiteturais adotadas, uma vez que a arquitetura define a organização dos componentes e as relações de dependência entre eles (Bass *et al.*, 2012; Sommerville, 2011). Dessa forma, a escolha de um estilo arquitetural exerce impacto direto na qualidade estrutural do software.

Dentre os estilos arquiteturais amplamente utilizados, destacam-se a arquitetura em camadas e a arquitetura hexagonal. A arquitetura em camadas organiza o sistema em níveis hierárquicos, nos quais cada camada possui responsabilidades específicas, promovendo separação de responsabilidades (Fowler, 2003). Por outro lado, a arquitetura hexagonal, também conhecida como Ports and Adapters, propõe o isolamento do núcleo da aplicação em relação às dependências externas, favorecendo maior independência tecnológica e testabilidade (Cockburn, 2005). Embora ambos os estilos sejam amplamente discutidos na literatura e utilizados na prática, seus impactos na modularidade nem sempre são avaliados de forma quantitativa.

Nesse cenário, métricas de software surgem como instrumentos fundamentais para a avaliação objetiva da qualidade estrutural. Métricas como coesão e acoplamento permitem quantificar propriedades internas do código, fornecendo base para análise de diferentes soluções (Fenton; Bieman, 2015). Em particular, métricas estruturais para sistemas orientados a objetos, como as propostas por Chidamber e Kemerer (1994), são amplamente utilizadas para avaliar atributos internos do software, tais como acoplamento, coesão e complexidade, os quais influenciam

diretamente a modularidade e a organização estrutural dos componentes.

Apesar da existência de diversos estudos que aplicam métricas de software para análise de sistemas, ainda há limitações na literatura no que diz respeito à comparação empírica entre diferentes estilos arquiteturais. Alguns trabalhos concentram-se em arquiteturas específicas, analisando seus impactos na modularidade em cenários controlados. Outros estudos realizam comparações entre estilos arquiteturais distintos, porém frequentemente baseados em revisões teóricas, sem a utilização de dados extraídos de sistemas reais.

Diante desse contexto, observa-se uma lacuna na realização de estudos experimentais que investiguem, de forma quantitativa, o impacto de diferentes estilos arquiteturais na modularidade de sistemas orientados a objetos, utilizando aplicações reais como objeto de análise. Assim, este trabalho propõe avaliar a modularidade de sistemas orientados a objetos por meio da aplicação de métricas estruturais, realizando um estudo comparativo entre arquitetura em camadas e arquitetura hexagonal. A análise será conduzida a partir de dois sistemas desenvolvidos previamente, permitindo a obtenção de evidências empíricas sobre os efeitos das decisões arquiteturais na qualidade estrutural do software.

Este trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, abordando conceitos de modularidade, arquitetura de software e métricas estruturais; o Capítulo 3, discorre sobre alguns trabalhos relacionados; o Capítulo 4 descreve a metodologia adotada; o Capítulo 5 discute os resultados obtidos; e, por fim, o Capítulo 6 apresenta as conclusões.

1.1 Objetivos

1.1.1 Objetivo Geral

Avaliar a modularidade de sistemas orientados a objetos por meio da aplicação de métricas estruturais em diferentes estilos arquiteturais.

1.1.2 Objetivos Específicos

- Avaliar a modularidade dos sistemas analisados com base nas métricas estruturais selecionadas;
- Comparar a modularidade entre os diferentes estilos arquiteturais analisados;
- Analisar a relação entre decisões arquiteturais e qualidade estrutural do software.

2 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica constitui a base conceitual necessária para compreensão dos temas abordados nesta pesquisa. Nesse contexto, este capítulo apresenta os principais conceitos relacionados à arquitetura de software, modularidade e qualidade estrutural de sistemas orientados a objetos. Também são discutidos os estilos arquiteturais analisados neste trabalho, bem como as métricas estruturais utilizadas para avaliar quantitativamente atributos relacionados ao acoplamento, à coesão e à complexidade dos sistemas estudados.

2.1 Arquitetura de Software

A arquitetura de software está relacionada à forma como um sistema é estruturado e compreendido em um nível mais alto de abstração. Segundo Bass *et al.* (2012), a arquitetura de um sistema corresponde ao conjunto de estruturas necessárias para raciocinar sobre ele, incluindo seus elementos de software, as relações estabelecidas entre esses elementos e as propriedades associadas a ambos.

Essa definição evidencia que a arquitetura não é composta por uma única estrutura, mas por diferentes estruturas que permitem analisar o sistema sob perspectivas distintas. De acordo com Bass *et al.* (2012), uma estrutura pode ser compreendida como um conjunto de elementos ligados por algum tipo de relação. Assim, a arquitetura permite observar como os elementos do sistema se organizam e como se relacionam, omitindo detalhes internos que não são relevantes para a compreensão da estrutura geral.

De forma complementar, Fowler (2003) destaca que o termo arquitetura não possui uma definição única amplamente aceita, mas geralmente envolve dois aspectos recorrentes: a decomposição do sistema em partes de alto nível e as decisões consideradas difíceis de modificar posteriormente. Nesse sentido, a arquitetura representa não apenas a organização estrutural do software, mas também um conjunto de decisões que influenciam a forma como o sistema pode ser compreendido, mantido e evoluído.

Em determinados contextos, os elementos arquiteturais são organizados de maneiras recorrentes para solucionar problemas comuns de projeto. Essas composições, descritas na literatura como padrões arquiteturais, fornecem estratégias reutilizáveis para estruturar sistemas de software (Bass *et al.*, 2012; Fowler, 2003). Entre os estilos relevantes para esta pesquisa destacam-se a arquitetura em camadas e a arquitetura hexagonal, que serão discutidas nas

subseções seguintes.

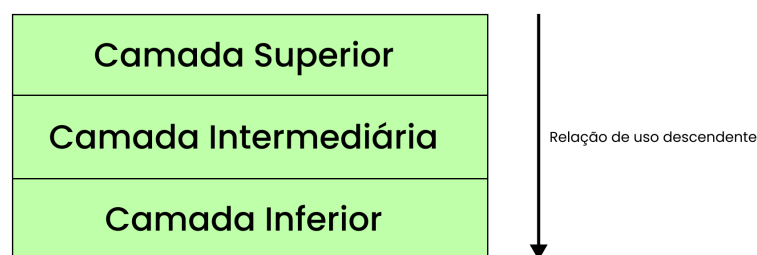
2.1.1 Arquitetura em Camadas

A arquitetura em camadas é um padrão arquitetural no qual os elementos do sistema são organizados em níveis de responsabilidade, chamados camadas. Segundo Bass *et al.* (2012), esse padrão emerge quando a relação de uso entre os elementos de software ocorre de forma unidirecional, de modo que uma camada utiliza os serviços fornecidos pelas camadas inferiores.

De acordo com Bass *et al.* (2012), uma camada corresponde a um conjunto coerente de funcionalidades relacionadas. Em uma estrutura estritamente em camadas, cada camada deve utilizar apenas os serviços oferecidos pela camada imediatamente inferior. Essa organização estabelece uma ordem clara de dependência entre as partes do sistema, favorecendo a separação de responsabilidades e a definição de interfaces públicas entre os níveis da aplicação.

O padrão em camadas tem como objetivo permitir que diferentes partes do sistema sejam desenvolvidas e evoluídas com menor interação direta entre si. Para isso, o software é segmentado em unidades que agrupam módulos responsáveis por oferecer um conjunto coeso de serviços. A Figura 1 apresenta uma representação simplificada desse padrão, na qual a camada superior pode utilizar os serviços da camada imediatamente inferior, mantendo uma relação de dependência orientada de cima para baixo.

Figura 1 – Representação do padrão arquitetural em camadas



Fonte: Elaborado pelo autor (2026), com base em Bass *et al.* (2012).

Na prática, existem variações desse padrão. Embora a estrutura estrita permita apenas o uso da camada imediatamente inferior, Bass *et al.* (2012) observa que alguns sistemas permitem o acesso a camadas inferiores não adjacentes, situação conhecida como *layer bridging*. Entretanto, a ocorrência frequente desse tipo de acesso pode comprometer benefícios associados ao padrão, como modificabilidade e portabilidade, pois aumenta o acoplamento entre partes que

deveriam permanecer separadas.

Embora a arquitetura em camadas seja definida principalmente pela separação de responsabilidades e pela relação de uso entre camadas, sua implementação pode empregar mecanismos de abstração para reduzir dependências diretas entre seus elementos. Conforme discutido por Bass *et al.* (2012), as camadas expõem seus serviços por meio de interfaces públicas, permitindo que uma camada utilize funcionalidades fornecidas por outra sem depender de seus detalhes internos de implementação. Assim, o uso de interfaces entre camadas não descaracteriza a arquitetura em camadas, mas representa uma forma de controlar o acoplamento entre seus elementos.

Dessa forma, a arquitetura em camadas organiza o software por meio de níveis de responsabilidade e relações de dependência controladas. Essa característica torna o padrão relevante para estudos sobre modularidade, uma vez que a forma como as camadas são estruturadas influencia diretamente o acoplamento entre os componentes e a separação de responsabilidades no sistema.

2.1.2 Arquitetura Hexagonal

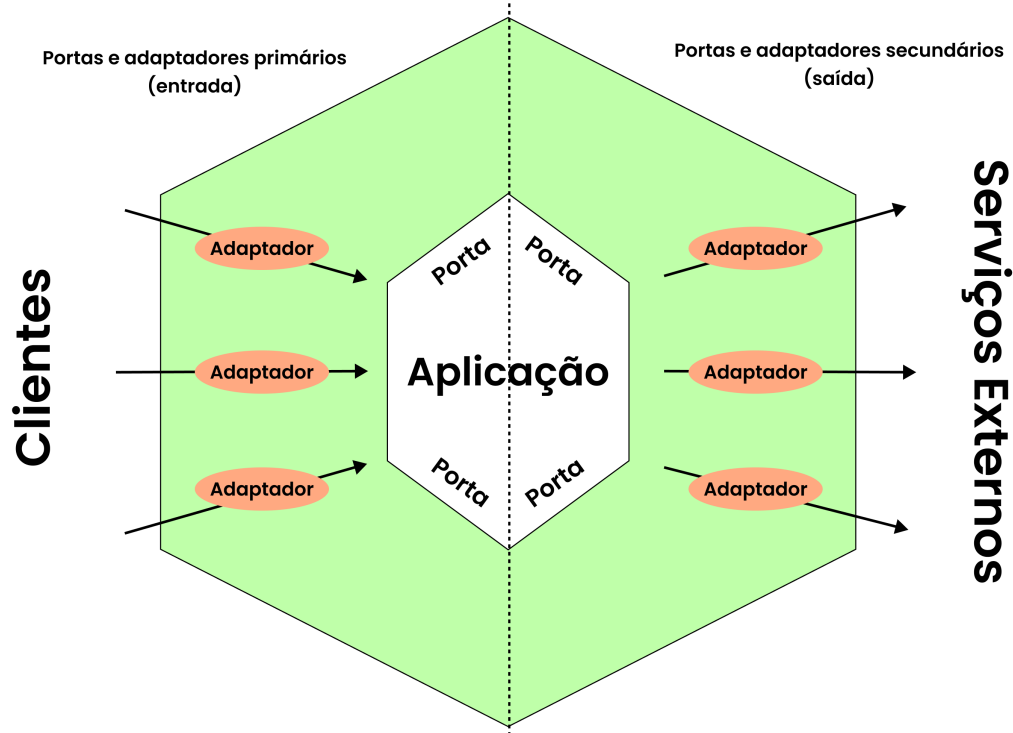
A arquitetura hexagonal, também conhecida como *Ports and Adapters*, foi proposta por Cockburn (2005) com o objetivo de permitir que uma aplicação seja desenvolvida e testada de forma isolada em relação às tecnologias externas utilizadas em sua execução. Nesse modelo, a aplicação deve ser capaz de interagir com diferentes mecanismos externos, como interfaces de usuário, bancos de dados, testes automatizados, scripts ou outros sistemas, sem que suas regras internas dependam diretamente dessas tecnologias (Cockburn, 2005).

Segundo Cockburn (2005), a motivação central desse padrão está relacionada à necessidade de evitar o acoplamento entre a lógica da aplicação e os elementos externos ao sistema. O autor destaca que problemas semelhantes ocorrem tanto quando regras de negócio são incorporadas à interface com o usuário quanto quando a lógica da aplicação passa a depender diretamente de bancos de dados ou serviços externos. Nesses casos, alterações tecnológicas podem afetar partes centrais do sistema, dificultando testes e a substituição de mecanismos externos.

A solução proposta pela arquitetura hexagonal consiste em estabelecer uma separação entre o interior e o exterior da aplicação. O núcleo da aplicação comunica-se com elementos externos por meio de portas, que representam interfaces ou protocolos de interação. Os adapta-

dores, por sua vez, são responsáveis por converter chamadas, mensagens ou sinais específicos de uma tecnologia externa para uma forma compreensível pela aplicação, e vice-versa (Cockburn, 2005). A Figura 2 apresenta uma representação desse padrão.

Figura 2 – Representação do padrão arquitetural hexagonal



Fonte: Elaborado pelo autor (2026), com base em Cockburn (2005).

Nesse padrão, o termo “hexagonal” não indica que a aplicação deve possuir exatamente seis lados ou seis interfaces. De acordo com Cockburn (2005), o uso do hexágono busca representar visualmente a existência de múltiplas portas ao redor da aplicação, destacando a separação entre o núcleo interno e os elementos externos. Assim, diferentes tipos de adaptadores podem ser conectados às portas da aplicação, permitindo substituir uma interface gráfica por testes automatizados ou trocar um banco de dados real por uma implementação em memória, sem alterar as regras internas do sistema (Cockburn, 2005).

Em implementações orientadas a objetos, as portas podem ser representadas por interfaces de programação que definem os serviços oferecidos ou requeridos pelo núcleo da aplicação. Os adaptadores implementam essas portas e conectam a aplicação a tecnologias específicas, como interfaces gráficas, APIs, mecanismos de persistência ou serviços externos (Cockburn, 2005). Essa organização favorece a independência tecnológica, pois as regras internas da aplicação permanecem separadas dos detalhes de infraestrutura e integração.

A arquitetura hexagonal diferencia-se da arquitetura em camadas por enfatizar a rela-

ção entre o interior e o exterior da aplicação, em vez de representar o sistema apenas como uma sequência vertical de camadas (Cockburn, 2005). Enquanto a arquitetura em camadas organiza o software em níveis hierárquicos de responsabilidade, a arquitetura hexagonal concentra-se na proteção do núcleo da aplicação contra dependências tecnológicas externas. Essa característica torna o padrão relevante para estudos sobre modularidade, uma vez que a forma como portas e adaptadores são definidos pode influenciar as dependências entre componentes e a separação entre regras de negócio e infraestrutura.

2.2 Modularidade em Software

A modularidade está diretamente relacionada à forma como um sistema é dividido em partes menores e mais gerenciáveis (Parnas, 1972). Em sistemas modulares, as funcionalidades são organizadas em unidades relativamente independentes, permitindo que cada módulo desempenhe responsabilidades específicas e possa ser compreendido, desenvolvido e modificado com menor impacto sobre os demais componentes do sistema.

Um dos trabalhos pioneiros sobre modularização foi apresentado por Parnas (1972), que argumenta que a decomposição de sistemas em módulos deve ser realizada de forma a minimizar a propagação de mudanças entre diferentes partes do software. Segundo o autor, uma estrutura modular adequada reduz a complexidade do desenvolvimento e facilita atividades de manutenção, evolução e correção de defeitos.

De acordo com Sommerville (2011), sistemas complexos podem ser organizados em componentes com responsabilidades específicas, favorecendo sua compreensão e reduzindo a complexidade do desenvolvimento. Essa decomposição também contribui para a reutilização de funcionalidades, uma vez que componentes bem definidos podem ser empregados em diferentes contextos com menor necessidade de modificações.

A importância da modularidade está associada à capacidade de controlar a complexidade inerente ao desenvolvimento de software. Segundo Pressman (2010), sistemas de grande porte tornam-se progressivamente mais difíceis de compreender e modificar quando apresentam dependências excessivas entre seus componentes. Nesse contexto, a modularização surge como uma estratégia para organizar responsabilidades e limitar o impacto das mudanças realizadas ao longo do ciclo de vida do software.

Embora o conceito de modularidade seja amplamente discutido na literatura, sua avaliação objetiva não é uma tarefa trivial. Conforme discutido por Fenton e Bieman (2015),

atributos relacionados à qualidade interna do software frequentemente apresentam natureza abstrata e, por esse motivo, podem ser avaliados por meio de indicadores quantitativos. Dessa forma, características associadas à modularidade são frequentemente investigadas por intermédio de métricas estruturais capazes de mensurar propriedades observáveis do código-fonte.

Entre os atributos frequentemente associados à avaliação da modularidade e da qualidade estrutural de sistemas orientados a objetos destacam-se o acoplamento, a coesão e a complexidade estrutural (Pressman, 2010; Fenton; Bieman, 2015). Assim, esses atributos serão discutidos nas subseções seguintes pois permitem analisar diferentes aspectos da organização estrutural do software.

2.2.1 Acoplamento

O acoplamento representa o grau de dependência existente entre módulos, componentes ou classes de um sistema de software. Em sistemas orientados a objetos, essa dependência ocorre quando uma classe estabelece relacionamentos com outras classes para utilizar seus serviços, trocar dados ou acessar elementos necessários à implementação de suas responsabilidades (Pressman, 2010).

Segundo Pressman (2010), à medida que componentes se tornam mais interdependentes, o grau de acoplamento aumenta. Como consequência, alterações realizadas em um componente tendem a produzir impactos em outros componentes relacionados.

Embora algum nível de acoplamento seja inevitável em sistemas de software, uma vez que seus componentes precisam se comunicar para executar suas funcionalidades, Pressman (2010) destaca que o projeto deve buscar reduzi-lo sempre que possível. Sistemas com baixo acoplamento favorecem a independência entre módulos, limitam a propagação de mudanças e facilitam a reutilização dos componentes.

O conceito de baixo acoplamento também está relacionado ao princípio da ocultação de informações (*information hiding*) proposto por Parnas (1972). Segundo esse princípio, os módulos devem expor apenas as informações estritamente necessárias para sua utilização, preservando seus detalhes internos de implementação. Dessa forma, modificações realizadas em um componente tendem a produzir menor impacto sobre os demais elementos do sistema.

O acoplamento constitui, portanto, um atributo importante para a organização de sistemas modulares, uma vez que influencia a independência entre componentes e a propagação de mudanças ao longo do software. Entretanto, a qualidade estrutural de um sistema não depende

apenas das relações estabelecidas entre seus módulos, mas também da organização interna de cada um deles. Esse aspecto é representado pelo conceito de coesão, discutido na subseção seguinte.

2.2.2 Coesão

A coesão refere-se ao grau de relacionamento existente entre os elementos internos de um módulo ou classe. Em sistemas orientados a objetos, um componente é considerado coeso quando encapsula atributos e operações fortemente relacionados entre si e direcionados a um mesmo propósito (Pressman, 2010).

Segundo Pressman (2010), a coesão constitui uma característica desejável em projetos de software, pois componentes cujos elementos estão fortemente relacionados tendem a ser mais simples de implementar, testar e manter. Em contrapartida, componentes que agrupam funcionalidades pouco relacionadas costumam apresentar maior complexidade e dificultam atividades de manutenção e evolução do sistema.

De acordo com Bass *et al.* (2012), a coesão está diretamente relacionada ao grau de associação entre as responsabilidades atribuídas a um módulo. Os autores descrevem esse atributo como uma medida da “unidade de propósito” (*unity of purpose*) de um componente, indicando que módulos altamente coesos tendem a concentrar responsabilidades relacionadas, reduzindo a probabilidade de que uma mudança em uma responsabilidade afete funcionalidades distintas.

Assim como o acoplamento descreve as dependências existentes entre diferentes componentes, a coesão concentra-se na organização interna de cada módulo. Dessa forma, sistemas modulares buscam simultaneamente reduzir as dependências entre componentes e concentrar responsabilidades relacionadas em unidades bem definidas.

Nesse contexto, a coesão constitui um atributo relevante para avaliação da modularidade de sistemas de software, complementando a análise do acoplamento ao considerar a forma como responsabilidades e funcionalidades são organizadas no interior de cada componente. Além disso, a avaliação da qualidade estrutural também envolve a análise da complexidade resultante da organização e das relações estabelecidas entre os elementos do sistema.

2.2.3 *Complexidade*

Neste trabalho, a complexidade é compreendida sob uma perspectiva estrutural, estando relacionada à dificuldade de compreender a organização de um sistema de software e as relações estabelecidas entre seus elementos (Pressman, 2010; Fenton; Bieman, 2015). Em sistemas orientados a objetos, essa complexidade pode ser observada a partir da forma como classes, métodos e dependências se organizam e interagem ao longo da estrutura do sistema.

Segundo Pressman (2010), existem diferentes formas de compreender a complexidade de software. No contexto de projetos orientados a objetos, a complexidade pode ser analisada a partir de características estruturais, especialmente pela forma como as classes se relacionam entre si. Dessa maneira, sistemas com maior quantidade de elementos interdependentes tendem a exigir maior esforço de compreensão, manutenção e evolução.

De forma complementar, Fenton e Bieman (2015) associa a complexidade estrutural à complexidade das conexões entre os elementos de um modelo de sistema. Para os autores, essa complexidade depende do número de ligações existentes entre os elementos, indicando que sistemas com maior quantidade de relações estruturais tendem a apresentar maior complexidade interna. Assim, a análise da complexidade estrutural permite observar não apenas a quantidade de elementos presentes no sistema, mas também como esses elementos estão conectados.

No contexto desta pesquisa, a complexidade é analisada a partir de propriedades observáveis no código-fonte dos sistemas. Dessa forma, não se pretende avaliar a complexidade algorítmica ou o desempenho das aplicações, mas sim características relacionadas à organização das classes, à quantidade de operações e ao potencial de interação entre os componentes.

Assim, a complexidade estrutural complementa os conceitos de acoplamento e coesão na avaliação da qualidade estrutural do software. Enquanto o acoplamento considera as dependências entre classes e a coesão considera a organização interna de cada módulo, a complexidade contribui para compreender o esforço necessário para analisar, modificar e evoluir os elementos estruturais do sistema.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta trabalhos relacionados ao tema desta pesquisa, com foco em estudos que investigam modularidade, qualidade arquitetural e organização de sistemas orientados a objetos. A análise desses trabalhos tem como objetivo identificar abordagens utilizadas na literatura para avaliação da qualidade estrutural de sistemas, bem como compreender como diferentes estilos arquiteturais impactam atributos relacionados à manutenção, evolução e organização do software.

Os trabalhos selecionados abordam diferentes perspectivas sobre o problema investigado. Alguns concentram-se na aplicação de métricas estruturais para avaliação da qualidade de sistemas orientados a objetos, enquanto outros analisam os impactos de diferentes arquiteturas de software em atributos como escalabilidade, manutenibilidade e acoplamento. Entretanto, observa-se que muitos desses estudos são fundamentados predominantemente em revisões bibliográficas ou análises conceituais, havendo menor quantidade de pesquisas empíricas voltadas à aplicação quantitativa de métricas estruturais em sistemas reais desenvolvidos sob diferentes estilos arquiteturais.

Nesse contexto, os trabalhos apresentados a seguir contribuem para fundamentar teoricamente esta pesquisa e auxiliar na identificação de lacunas existentes na literatura, especialmente no que se refere à avaliação quantitativa da modularidade em arquiteturas distintas.

3.1 Avaliação de Conectividade em Sistemas Orientados por Objetos

O trabalho de Ferreira (2006) propõe o Modelo de Avaliação de Conectividade de Sistemas Orientados por Objetos (MACSOO), fundamentado na tese de que a conectividade entre módulos é o fator preponderante para determinar a estabilidade e a manutenibilidade de um software. A pesquisa parte do princípio de que a manutenção consome a maior parte do custo total de um sistema e que estruturas mais flexíveis, com menor interdependência entre suas partes, facilitam esse processo.

Segundo a autora, a orientação por objetos, embora favoreça a modularidade, não garante por si só uma estrutura de alta qualidade se não houver um controle rigoroso sobre as conexões entre classes. O estudo defende que o impacto de uma alteração no sistema é diretamente proporcional ao seu grau de conectividade, e que o "apodrecimento" do design de software ocorre quando a rigidez estrutural impede modificações sem gerar efeitos colaterais em

cascata.

Para operacionalizar a avaliação, o trabalho identifica e classifica métricas estruturais clássicas, com destaque para o conjunto CK (Chidamber e Kemerer) e o conjunto MOOD (Metrics for Object Oriented Design), selecionando indicadores como CBO (*Coupling Between Object Classes*), LCOM (*Lack of Cohesion in Methods*) e COF (*Coupling Factor*). Além disso, a pesquisa contribuiu com o desenvolvimento da ferramenta *Connecta*, capaz de extrair essas métricas de sistemas desenvolvidos em Java e aplicar uma adaptação do modelo de estabilidade de Myers para prever o esforço de manutenção.

A validação do modelo proposto por Ferreira (2006) foi realizada por meio de experimentos em sistemas reais e acadêmicos, demonstrando empiricamente que a dificuldade de manutenção cresce à medida que o acoplamento aumenta e a coesão diminui. Entretanto, embora o estudo forneça uma base sólida para a análise de conectividade em sistemas orientados por objetos de forma geral, ele não se aprofunda na comparação de como estilos arquiteturais específicos, como o em camadas ou o hexagonal, moldam esses indicadores estruturais.

Nesse contexto, o presente trabalho fundamenta-se nas premissas metodológicas de Ferreira (2006) ao adotar métricas estruturais consagradas para avaliar a modularidade de software. Esta pesquisa diferencia-se e avança em relação ao trabalho relacionado ao aplicar essa análise quantitativa em um estudo comparativo direto entre a arquitetura em camadas e a arquitetura hexagonal. Enquanto Ferreira concentra-se na conectividade como um indicador de qualidade para sistemas OO genéricos, este estudo busca investigar como a escolha entre esses dois estilos arquiteturais específicos influencia o acoplamento e a coesão.

3.2 Os efeitos da arquitetura dirigida a eventos na modularidade de software

O trabalho de Lazzari e Farias (2022) apresenta um estudo empírico exploratório que busca compreender os efeitos da arquitetura dirigida a eventos na modularização de software em comparação ao estilo arquitetural REST. A pesquisa fundamenta-se na análise de duas versões funcionalmente idênticas de uma aplicação real, evoluídas através de cinco cenários de mudança nos quais novas funcionalidades foram adicionadas. O estudo utiliza um conjunto de dez métricas estruturais para quantificar variáveis críticas da modularidade, incluindo separação de interesses (SoC), acoplamento, coesão, complexidade e tamanho.

Segundo os resultados reportados pelos autores, a arquitetura dirigida a eventos (implementada com Apache Kafka) demonstrou superioridade na separação de interesses, apre-

sentando melhores índices de modularização funcional tanto em nível de componentes quanto de linhas de código. Esse achado sugere que o modelo de eventos favorece a independência dos serviços, facilitando a evolução do sistema sem a degradação da qualidade funcional. Por outro lado, a pesquisa revelou que essa flexibilidade introduz uma "penalidade" estrutural: a arquitetura dirigida a eventos apresentou maior acoplamento e complexidade se comparada ao REST, além de um aumento significativo no tamanho do código devido às necessidades de configuração de infraestrutura e classes auxiliares.

O trabalho contribui para a área ao prover evidências quantitativas sobre os trade-offs de um estilo arquitetural moderno em cenários reais de evolução. No entanto, os próprios autores reconhecem limitações no estudo, como a análise centrada em uma única aplicação alvo e a elevada curva de aprendizado exigida pela arquitetura de eventos, o que pode influenciar a implementação e a extração das métricas. Dessa forma, embora o trabalho apresente uma base experimental robusta, os resultados focam especificamente na dicotomia entre comunicação síncrona e assíncrona, sem aprofundar-se em estilos organizacionais internos do monólito.

Nesse contexto, o presente trabalho aproxima-se da proposta de Lazzari e Farias (2022) ao adotar uma estratégia de pesquisa experimental baseada na comparação de arquiteturas através de métricas estruturais. No entanto, esta pesquisa diferencia-se por investigar o impacto de estilos arquiteturais voltados à organização interna do software em vez de focar na comunicação dirigida a eventos.

3.3 Comparação entre as arquiteturas de software monolítica e de microserviços

O trabalho de Vale (2025) investiga o debate entre as arquiteturas monolítica e de microserviços, focando na transição da análise teórica para a validação empírica. O estudo utiliza uma pesquisa quantitativa de natureza experimental para mensurar os trade-offs de cada abordagem em um ambiente controlado. A metodologia consistiu na implementação de dois protótipos funcionalmente idênticos de um sistema de e-commerce (CRUD), desenvolvidos com o ecossistema Node.js e containerizados com Docker, para garantir que as diferenças observadas fossem atribuídas exclusivamente às variações arquiteturais.

Segundo os resultados reportados pelo autor, a arquitetura monolítica demonstrou ser objetivamente mais eficiente em todas as métricas de performance local e custo operacional. O achado mais significativo revelou uma penalidade de desempenho de 147% na latência de operações que exigiam orquestração entre múltiplos domínios na arquitetura de microserviços,

em comparação ao monólito. Além disso, o estudo quantificou o "imposto de memória" da distribuição, apontando que os microsserviços consumiram 271% mais memória RAM em estado de repouso devido à sobrecarga de manter múltiplos processos e contêineres ativos.

No que tange à complexidade, o trabalho apresenta uma análise qualitativa indicando que a arquitetura de microsserviços impõe uma carga cognitiva e operacional superior. Isso foi evidenciado pelo número de artefatos de configuração necessários para a implantação, que foi três vezes maior do que o exigido para o monólito. O autor conclui que, embora os microsserviços ofereçam benefícios teóricos de escalabilidade, para sistemas de baixa complexidade ou em fases iniciais, a arquitetura monolítica é fortemente suportada pela eficiência de recursos e velocidade de entrega.

Embora este TCC compartilhe com Vale (2025) o uso de uma abordagem quantitativa para analisar trade-offs arquiteturais, a estratégia de comparação diverge em sua fundamentação. Enquanto o referido autor utiliza protótipos funcionalmente idênticos para isolar variáveis de desempenho como latência e consumo de hardware, esta pesquisa foca na análise da modularidade interna por meio de métricas estruturais aplicadas a sistemas reais com funcionalidades distintas. Assim, enquanto Vale (2025) quantifica custos de infraestrutura em modelos distribuídos, este estudo busca identificar como diferentes padrões organizacionais moldam os indicadores de acoplamento e coesão, independentemente da paridade funcional, visando avaliar a sustentabilidade e a estabilidade de cada design.

3.4 Análise comparativa entre arquiteturas de software: monolítica, modular e microsserviços

O trabalho de Cintra *et al.* (2025) realiza uma análise comparativa entre as arquiteturas monolítica, monólito modular e microsserviços, buscando identificar características, vantagens, limitações e cenários de aplicação de cada abordagem arquitetural. O estudo é fundamentado em revisão bibliográfica e análise qualitativa, considerando atributos relevantes da Engenharia de Software, como escalabilidade, manutenibilidade, resiliência e complexidade operacional.

Segundo os autores, a arquitetura monolítica apresenta vantagens relacionadas à simplicidade de desenvolvimento, implantação e gerenciamento, especialmente em sistemas de pequeno e médio porte. Entretanto, à medida que o sistema cresce, problemas relacionados ao aumento do acoplamento e à dificuldade de manutenção tornam-se mais evidentes. Em

contrapartida, a arquitetura de microsserviços oferece maior flexibilidade, escalabilidade independente e isolamento de falhas, embora introduza desafios associados à comunicação distribuída, monitoramento e maior complexidade operacional (Cintra *et al.*, 2025).

O trabalho também destaca o monólito modular como uma alternativa intermediária entre os dois extremos arquiteturais. De acordo com os autores, essa abordagem busca manter a simplicidade estrutural do monólito tradicional, porém com maior organização interna e separação de responsabilidades, favorecendo a manutenibilidade e facilitando uma possível migração incremental para microsserviços. Além disso, o estudo propõe um guia heurístico para apoio à tomada de decisão arquitetural, utilizando pesos e pontuações atribuídos a diferentes atributos de qualidade.

Apesar das contribuições apresentadas, os próprios autores reconhecem limitações importantes na pesquisa. O estudo foi conduzido predominantemente com base em revisão bibliográfica e análise qualitativa, não sendo realizados experimentos empíricos ou medições quantitativas em sistemas reais (Cintra *et al.*, 2025). Dessa forma, embora o trabalho apresente uma discussão abrangente sobre os impactos das arquiteturas analisadas, não há validação quantitativa baseada em métricas estruturais extraídas diretamente de implementações concretas.

Nesse contexto, o presente trabalho aproxima-se da proposta de Cintra *et al.* (2025) ao investigar como diferentes estilos arquiteturais influenciam atributos relacionados à qualidade estrutural do software. No entanto, esta pesquisa diferencia-se por adotar uma abordagem quantitativa baseada na aplicação de métricas estruturais em sistemas reais desenvolvidos sob diferentes arquiteturas. Enquanto o trabalho relacionado concentra-se em análises conceituais e heurísticas sobre arquiteturas monolíticas, modulares e distribuídas, este estudo busca avaliar empiricamente aspectos relacionados à modularidade, como acoplamento, coesão e estabilidade estrutural, comparando arquiteturas em camadas e arquitetura hexagonal.

3.5 Um estudo sobre a relação entre qualidade e arquitetura de software

O trabalho de Tsuruta (2010) investiga como a arquitetura de software atua como fator determinante para a qualidade final do produto, analisando quatro abordagens distintas para sua especificação: clássica, orientada a objetos, orientada a atributos e orientada a busca. A pesquisa fundamenta-se na premissa de que a qualidade não reside apenas na ausência de defeitos funcionais, mas na capacidade do sistema de satisfazer requisitos não-funcionais críticos, como a manutenibilidade e a capacidade de modificação.

O autor destaca que o design arquitetural deve ser guiado pelos atributos de qualidade mais importantes para o sucesso do projeto. Para viabilizar essa análise, o estudo utiliza a métrica de arquitetura denominada Qualidade de Modularização (TurboMQ), que quantifica a interdependência entre componentes ao calcular a razão entre relacionamentos internos e externos em um gráfico de dependência de módulos (MDG). Um dos diferenciais da pesquisa é a aplicação da Engenharia de Software Baseada em Busca (SBSE), utilizando Algoritmos Genéticos para encontrar automaticamente o particionamento ideal de responsabilidades em subsistemas que maximize a modularidade.

Embora forneça uma base teórica robusta sobre modelos de qualidade (como a ISO 9126) e demonstre a eficácia de técnicas meta-heurísticas na otimização estrutural, o estudo de Tsuruta (2010) concentra-se em uma aplicação experimental com responsabilidades hipotéticas para validar o algoritmo. Dessa forma, a pesquisa não realiza uma comparação empírica direta entre padrões arquiteturais consolidados, como o estilo em camadas ou o hexagonal, em implementações de software completas. Sob a ótica deste TCC, a investigação aqui conduzida compartilha com Tsuruta (2010) a visão de que a organização dos componentes em subsistemas (ou camadas) é a chave para a estabilidade estrutural. Entretanto, este estudo diverge ao adotar uma análise empírica baseada na extração de métricas de sistemas reais em vez de buscar a otimização automatizada por algoritmos de busca. Enquanto o referido autor explora o TurboMQ para particionamento lógico, esta pesquisa utiliza métricas de acoplamento e coesão para avaliar como os estilos Hexagonal e em Camadas moldam a modularidade do código-fonte.

3.6 Síntese e lacunas

Após a análise individual dos trabalhos, esta seção apresenta uma visão consolidada das pesquisas selecionadas. A Tabela 1 sistematiza os principais atributos e metodologias, facilitando a identificação de lacunas na literatura.

A revisão dos trabalhos relacionados demonstra que a análise da qualidade de software tem migrado de discussões puramente teóricas para abordagens empíricas fundamentadas em métricas quantitativas. A literatura analisada revela uma forte tendência na utilização de experimentos controlados para validar os trade-offs entre diferentes estilos arquiteturais, como observado nos estudos comparativos entre monólitos e microsserviços. No que diz respeito à modularidade interna, as métricas estruturais consagradas, como o conjunto CK e MOOD, permanecem como os principais instrumentos para quantificar o acoplamento, a coesão e a

Tabela 1 – Comparativo de abordagens dos trabalhos relacionados

Autor (Ano)	Objetivo Principal	Metodologia	Estilos Arquiteturais	Foco na Mod.
Ferreira (2006)	Propor o modelo MACSOO para avaliar conectividade.	Experimental (Métricas CK/MOOD em Java).	Sistemas OO genéricos.	Alto
Tsuruta (2010)	Analisar a relação entre qualidade e arquitetura.	Otimização via Busca (Algoritmos Genéticos).	Responsabilidades hipotéticas (TurboMQ).	Médio
Lazzari (2022)	Investigar efeitos da EDA na modularização.	Experimental (Cenários de evolução).	Event-Driven vs. REST.	Alto
Cintra (2025)	Identificar trade-offs para suporte à decisão.	Revisão Bibliográfica Sistemática.	Monolítica, Modular e Microserviços.	Médio
Vale (2025)	Quantificar custos de desempenho e infraestrutura.	Experimental (Protótipos equivalentes).	Monolítica vs. Microserviços.	Baixo
Este Trabalho	Avaliar modularidade via métricas estruturais.	Experimental (Sistemas SISO e Roomie).	Camadas vs. Hexagonal.	Alto

Fonte: Elaborado pelo autor (2026).

conectividade de sistemas orientados a objetos.

Entretanto, apesar das contribuições apresentadas pelos trabalhos analisados, ainda existem lacunas importantes relacionadas à avaliação quantitativa da modularidade em diferentes estilos arquiteturais. A maior parte das investigações comparativas concentra-se em arquiteturas de macro-nível, como sistemas monolíticos e distribuídos (Cintra *et al.*, 2025; Vale, 2025), ou em diferentes estilos de comunicação, como REST e EDA (Lazzari; Farias, 2022). Há uma carência de estudos que comparem de forma quantitativa os padrões de organização interna de sistemas monolíticos, especificamente a arquitetura em camadas com a arquitetura hexagonal.

Em segundo lugar, identifica-se uma predominância de avaliações qualitativas ou baseadas em revisões bibliográficas que carecem de validação em implementações de software concretas (Cintra *et al.*, 2025). Mesmo trabalhos que exploram a otimização de atributos de qualidade, como o de Tsuruta (2010), frequentemente aplicam algoritmos a modelos hipotéticos ou responsabilidades isoladas, sem considerar a complexidade de sistemas completos. Além disso, enquanto alguns autores focam em métricas de infraestrutura como latência e consumo de hardware (Vale, 2025), a análise estrutural do código e seu impacto na modularidade é menos explorada em contextos comparativos diretos.

Este TCC propõe-se a preencher essas lacunas ao realizar um estudo experimental focado na modularidade estrutural de dois sistemas: o SISO (Arquitetura em Camadas) e o Roomie (Arquitetura Hexagonal). Diferentemente de abordagens que exigem paridade funcional entre os sistemas analisados, esta pesquisa aproxima-se da estratégia utilizada por Ferreira (2006), avaliando a qualidade do design por meio de métricas estruturais aplicadas a sistemas com funcionalidades distintas. Dessa forma, busca-se analisar como diferentes estilos arquiteturais influenciam atributos relacionados ao acoplamento e à coesão, produzindo evidências empíricas sobre os impactos dessas arquiteturas na modularidade de sistemas orientados a objetos.

4 METODOLOGIA

Este capítulo apresenta os procedimentos metodológicos adotados para a condução da pesquisa. Inicialmente, são descritas a classificação e a estratégia da investigação. Em seguida, são apresentados os objetos de estudo, as métricas utilizadas para avaliação da modularidade, os procedimentos de coleta dos dados e as etapas do experimento proposto.

4.1 Classificação da Pesquisa

Segundo Gil (2008), as pesquisas podem ser classificadas quanto à natureza, aos objetivos, à abordagem do problema e aos procedimentos técnicos. Sob essa perspectiva, este trabalho caracteriza-se como uma pesquisa aplicada, descritiva, de abordagem quantitativa e conduzida por meio de um estudo experimental.

A pesquisa é classificada como aplicada por buscar produzir conhecimento com potencial de utilização prática na avaliação da modularidade e da qualidade estrutural de sistemas de software. Além disso, possui caráter descritivo, uma vez que busca identificar, mensurar e comparar características estruturais observadas em sistemas desenvolvidos segundo diferentes estilos arquiteturais.

A abordagem adotada é quantitativa, fundamentando-se na coleta e análise de dados numéricos obtidos por meio de métricas estruturais extraídas do código-fonte dos sistemas analisados. Por fim, o trabalho configura-se como um estudo experimental, pois compara métricas obtidas a partir de dois sistemas implementados segundo arquiteturas distintas, buscando identificar diferenças relacionadas à modularidade estrutural.

4.2 Estratégia de Pesquisa

A estratégia adotada nesta pesquisa consiste na extração e análise de métricas estruturais obtidas a partir de dois sistemas desenvolvidos segundo estilos arquiteturais distintos. Os valores observados serão comparados com o objetivo de identificar diferenças relacionadas ao acoplamento, à coesão e à complexidade estrutural das classes.

Conforme argumentam Fenton e Bieman (2015), métricas de software permitem transformar propriedades abstratas do projeto em indicadores quantitativos, possibilitando análises mais objetivas sobre qualidade interna e manutenibilidade. Dessa forma, a utilização de métricas estruturais permite investigar empiricamente características normalmente avaliadas de

maneira subjetiva.

A unidade de análise desta pesquisa corresponde às classes implementadas em cada sistema. A escolha desse nível de granularidade justifica-se pelo fato de que as métricas selecionadas foram originalmente concebidas para avaliação de sistemas orientados a objetos (Chidamber; Kemerer, 1994). Além disso, trabalhos relacionados analisados neste estudo demonstram a utilização de métricas estruturais como instrumento para avaliação de modularidade e qualidade arquitetural, seja por meio de métricas orientadas a objetos (Ferreira, 2006) ou de abordagens quantitativas aplicadas à comparação entre estilos arquiteturais (Lazzari; Farias, 2022; Tsuruta, 2010). Dessa forma, a análise em nível de classe é consistente com a literatura e adequada aos objetivos desta pesquisa.

4.3 Objetos de Estudo

Esta pesquisa utiliza como objetos de estudo dois sistemas desenvolvidos em contexto acadêmico durante disciplinas do curso de Engenharia de Software. Os sistemas foram implementados por equipes de estudantes, com participação do autor desta pesquisa, e foram selecionados por adotarem estilos arquiteturais distintos, possibilitando a investigação dos impactos dessas arquiteturas sobre atributos relacionados à modularidade de software.

O primeiro sistema analisado é o *SISO*, desenvolvido na disciplina de Projeto Integrado II. Trata-se de uma aplicação web voltada ao gerenciamento de clínicas odontológicas, oferecendo funcionalidades relacionadas ao controle de usuários, cadastro de pacientes, prontuário eletrônico, anamnese, gerenciamento de tratamentos e agendamentos. O sistema foi concebido utilizando uma arquitetura em camadas, modelo no qual as responsabilidades da aplicação são distribuídas em níveis específicos, favorecendo a separação de interesses e a organização estrutural do software (Fowler, 2003).

O segundo sistema analisado é o *Roomie*, desenvolvido na disciplina de Projeto Integrado III. O sistema consiste em uma aplicação móvel destinada a conectar pessoas que possuem vagas disponíveis em residências com indivíduos que buscam compartilhar moradia. Entre suas funcionalidades estão a publicação de anúncios, mecanismos de filtragem, sistema de compatibilidade entre usuários, troca de mensagens por chat e envio de notificações. Diferentemente do *SISO*, o *Roomie* foi estruturado segundo a arquitetura hexagonal, também conhecida como *Ports and Adapters*. De acordo com Cockburn (2005), esse estilo arquitetural busca isolar as regras de negócio dos detalhes tecnológicos por meio de interfaces bem definidas, reduzindo

o acoplamento entre domínio e infraestrutura.

Embora os sistemas pertençam a domínios de negócio distintos, ambos foram desenvolvidos utilizando Java e o framework Spring Boot em suas camadas de back-end, adotando princípios de orientação a objetos e práticas semelhantes de desenvolvimento. Dessa forma, a comparação proposta concentra-se na organização estrutural promovida pelos estilos arquiteturais adotados, e não nas funcionalidades implementadas ou no desempenho das aplicações.

Tabela 2 – Caracterização dos sistemas analisados

Característica	SISO	Roomie
Arquitetura	Em Camadas	Hexagonal
Classes	89	88
Interfaces	47	38
Enums	6	18

Fonte: Elaborado pelo autor (2026).

Observa-se que os sistemas apresentam quantidades semelhantes de classes, indicando porte estrutural comparável sob essa perspectiva. Essa característica contribui para a realização de uma análise comparativa focada nos efeitos dos estilos arquiteturais sobre métricas relacionadas à modularidade, reduzindo a influência de diferenças significativas de escala entre os projetos.

Os repositórios contendo os sistemas analisados encontram-se disponíveis para consulta e reprodução dos experimentos realizados neste trabalho, conforme apresentado no Apêndice A.

4.4 Coleta dos Dados

A coleta dos dados será realizada por meio de análise estática do código-fonte dos sistemas estudados. Nessa abordagem, as métricas são obtidas diretamente a partir da estrutura do software, sem a necessidade de execução da aplicação. Essa estratégia é compatível com o objetivo da pesquisa, que consiste em avaliar atributos internos relacionados à modularidade por meio de métricas estruturais (Fenton; Bieman, 2015).

Para extração das métricas será utilizada a ferramenta CK, desenvolvida por Mauricio Aniche. A ferramenta realiza a análise estática de projetos Java e gera relatórios contendo métricas estruturais em nível de classe e método (Aniche, 2015). O link para acesso a ferramenta CK encontra-se disponível no Apêndice A.

Nesta pesquisa serão utilizados exclusivamente os dados extraídos em nível de classe,

uma vez que o foco do estudo está na organização estrutural dos sistemas e não na complexidade individual de métodos específicos.

4.5 Métricas de Avaliação

As métricas utilizadas neste estudo seguem as definições clássicas propostas por Chidamber e Kemerer (Chidamber; Kemerer, 1994), enquanto a operacionalização dos cálculos segue a implementação fornecida pela ferramenta CK (Aniche, 2015).

As métricas selecionadas foram escolhidas por sua relação direta com atributos associados à modularidade, especialmente acoplamento, coesão e complexidade estrutural, sendo descritas na Tabela 3.

Embora o conjunto original de 1994 seja composto por seis métricas, este trabalho utiliza apenas quatro delas, excluindo propositalmente DIT e NOC. Essa decisão justifica-se pelo fato de DIT e NOC avaliarem predominantemente a genealogia e a hierarquia de herança das classes. Como o foco central deste TCC é a modularidade estrutural e o acoplamento entre camadas e portas, métricas ligadas à herança não trariam conclusões relevantes para a comparação dos estilos arquiteturais analisados.

Para complementar a análise, foram incorporadas as métricas Fan-In e Fan-Out. Apesar de não integrarem a suíte original de 1994, elas foram selecionadas por fornecerem uma visão granular das dependências de entrada e saída, o que é relevante para avaliar a comunicação entre os componentes de cada sistema.

Convém observar que as definições apresentadas na Tabela 3 apresentam variações em relação às proposições conceituais originais de Chidamber e Kemerer (1994). Tais distinções decorrem da operacionalização prática adotada pela ferramenta CK, que utiliza heurísticas de análise estática específicas para o ecossistema Java. Por exemplo, enquanto o CBO original é definido de forma ampla como o acoplamento entre classes, a ferramenta especifica exatamente quais referências (como variáveis locais e retornos) são contabilizadas e ignora propositalmente as bibliotecas padrão do Java para garantir maior precisão nos resultados. Da mesma forma, a métrica de coesão utilizada é o LCOM*, uma versão normalizada que difere do cálculo de interseção de conjuntos proposto inicialmente. Para os fins deste estudo, as definições operacionais da ferramenta serão as adotadas, garantindo a consistência entre a extração dos dados e a análise dos resultados.

Tabela 3 – Métricas utilizadas na avaliação da modularidade

Métrica	Descrição
CBO	Mede o número de dependências de uma classe com outras classes do sistema. Na ferramenta CK, considera qualquer tipo referenciado na classe (atributos, parâmetros, retornos e variáveis locais), exceto tipos da biblioteca padrão Java (Aniche, 2015).
LCOM*	Mede a falta de coesão entre métodos de uma classe. É uma métrica normalizada no intervalo de 0 a 1, em que valores próximos de 0 indicam maior coesão e valores próximos de 1 indicam menor coesão. A ferramenta CK disponibiliza a métrica LCOM* em sua forma normalizada. (Aniche, 2015).
RFC	Conta o número de métodos que podem ser potencialmente executados em resposta a uma mensagem recebida pela classe, incluindo métodos próprios e invocações a outros métodos. Na ferramenta CK, é obtido por análise estática de invocações únicas de métodos (Aniche, 2015).
WMC	Mede a complexidade de uma classe. Na ferramenta CK, é calculado como o número de instruções de ramificação presentes nos métodos da classe (Aniche, 2015).
Fan-In	Quantifica o número de dependências de entrada de uma classe, correspondendo à quantidade de classes que a referenciam por meio de atributos, acesso a membros ou invocação de métodos. Na ferramenta CK, considera todas as referências diretas à classe no código-fonte (Aniche, 2015).
Fan-Out	Quantifica o número de dependências de saída de uma classe, correspondendo à quantidade de outras classes referenciadas por ela por meio de atributos, instanciação de objetos ou invocação de métodos. Na ferramenta CK, considera todas as referências diretas feitas pela classe no código-fonte (Aniche, 2015).

Fonte: Elaborado pelo autor (2026).

4.6 Procedimento Experimental

O procedimento experimental será executado em três etapas.

Inicialmente, a ferramenta CK será executada individualmente nos sistemas SISO e Roomie para extração das métricas estruturais definidas nesta pesquisa. A ferramenta realizará a análise estática do código-fonte e produzirá uma planilha como saída contendo os valores das métricas para cada classe identificada nos projetos.

Na segunda etapa, os dados extraídos pela ferramenta CK serão processados por meio de um script desenvolvido em Python. O script será responsável pela seleção das métricas definidas na pesquisa, pelo cálculo das medidas de estatística descritiva e geração de visualizações

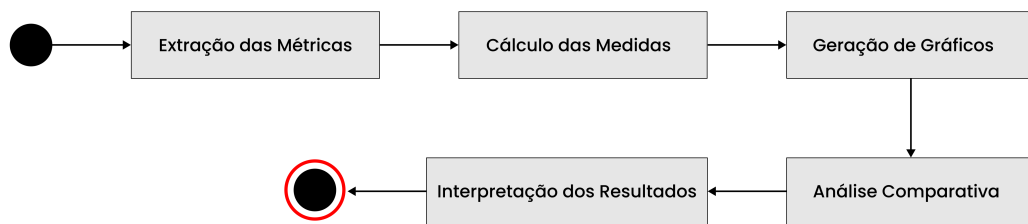
gráficas que auxiliem a interpretação e comparação dos resultados. Serão calculadas, para cada métrica, as medidas de média, mediana, desvio padrão, mínimo e máximo.

A utilização de recursos computacionais para tratamento dos dados busca reduzir erros de processamento manual e garantir a reprodutibilidade das análises realizadas.

Por fim, será realizada a análise comparativa dos resultados, buscando identificar diferenças nos indicadores de acoplamento, coesão e complexidade estrutural entre os sistemas analisados. A interpretação dos resultados será conduzida com base nos conceitos de modularidade discutidos na fundamentação teórica, permitindo avaliar como os diferentes estilos arquiteturais se refletem nas métricas estruturais consideradas neste estudo.

A Figura 3 apresenta uma visão geral das etapas que compõem o procedimento experimental adotado nesta pesquisa.

Figura 3 – Fluxo das etapas do procedimento experimental



Fonte: Elaborado pelo autor (2026).

4.7 Cronograma de Execução

Essa seção apresenta o cronograma de execução das atividades previstas para a realização do TCC II, conforme descrito na Tabela 4.

Tabela 4 – Cronograma de execução do TCC II

Atividade	Ago	Set	Out	Nov	Dez
Revisão e ajustes do TCC I	X				
Extração das métricas com a ferramenta CK		X			
Desenvolvimento do script de análise		X	X		
Tratamento dos dados e consolidação dos resultados			X		
Análise e interpretação dos resultados			X	X	
Redação dos resultados e discussão				X	
Elaboração da conclusão				X	
Revisão geral do trabalho				X	X
Adequação às normas da instituição					X
Entrega da versão final					X

Fonte: Elaborado pelo autor (2026).

5 RESULTADOS

6 CONCLUSÕES

REFERÊNCIAS BIBLIOGRÁFICAS

- ANICHE, M. **Java code metrics calculator (CK)**. [S. l.], 2015. Available in <https://github.com/mauricioaniche/ck/>.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 3. ed. [S. l.]: Addison-Wesley, 2012.
- CHIDAMBER, S. R.; KEMERER, C. F. A metrics suite for object oriented design. **IEEE Transactions on Software Engineering**, v. 20, n. 6, p. 476–493, 1994.
- CINTRA, L. R.; MARTINIANO, P. P.; BORGES, L. Análise comparativa entre arquiteturas de software monolítica, modular e microsserviços. **RESIGET**, v. 15, n. 1, p. 180–209, 2025.
- COCKBURN, A. **Hexagonal Architecture**. 2005. Acesso em: 24 abr. 2026. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>.
- FENTON, N.; BIEMAN, J. **Software Metrics: A Rigorous and Practical Approach**. 3. ed. [S. l.]: CRC Press, 2015.
- FERREIRA, K. A. M. **Avaliação de Conectividade em Sistemas Orientados por Objetos**. Dissertação (Dissertação (Mestrado em Ciência da Computação)) – Universidade Federal de Minas Gerais, Belo Horizonte, 2006.
- FOWLER, M. **Patterns of Enterprise Application Architecture**. [S. l.]: Addison-Wesley, 2003.
- GIL, A. C. **Métodos e Técnicas de Pesquisa Social**. 6. ed. São Paulo: Atlas, 2008.
- LAZZARI, L.; FARIAS, K. Os efeitos da arquitetura dirigida a eventos na modularidade de software: Um estudo exploratório. In: **Anais do Simpósio Brasileiro de Engenharia de Software (ou similar)**. São Leopoldo: [S. n.], 2022.
- PARNAS, D. L. On the criteria to be used in decomposing systems into modules. **Communications of the ACM**, v. 15, n. 12, p. 1053–1058, 1972.
- PRESSMAN, R. S. **Software Engineering: A Practitioner's Approach**. 7. ed. [S. l.]: McGraw-Hill, 2010.
- SOMMERVILLE, I. **Software Engineering**. 9. ed. [S. l.]: Pearson, 2011.
- TSURUTA, M. **Um estudo sobre a relação entre qualidade e arquitetura de software**. Dissertação (Dissertação (Mestrado em Administração)) – Universidade de São Paulo, São Paulo, 2010.
- VALE, P. T. P. do. Trabalho de Conclusão de Curso (Graduação em Engenharia da Computação), **Comparação entre as arquiteturas de software monolítica e de microsserviços**. Sobral: [S. n.], 2025.
- YOURDON, E.; CONSTANTINE, L. L. **Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design**. [S. l.]: Yourdon Press, 1978.

APÊNDICE A – FERRAMENTAS E REPOSITÓRIOS UTILIZADOS NA PESQUISA

Com o objetivo de favorecer a transparência e a reprodutibilidade da pesquisa, este apêndice apresenta os repositórios dos sistemas utilizados como objetos de estudo e a ferramenta empregada para extração das métricas estruturais.

A.1 Ferramenta de Extração de Métricas

A coleta das métricas será realizada por meio da ferramenta CK, desenvolvida para análise estática de projetos Java orientados a objetos.

- Ferramenta CK: <https://github.com/mauricioaniche/ck>

A.2 Repositórios dos Sistemas Analisados

Os sistemas utilizados nesta pesquisa encontram-se armazenados em repositórios Git, os quais servirão como fonte para a extração das métricas estruturais descritas na metodologia.

- SISO (Arquitetura em Camadas): <https://github.com/Werley-08/SISO>
- Roomie (Arquitetura Hexagonal): https://github.com/WendelRodriguesz/Roomie_Android